S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

Practical No. 2

Aim: To implement **Linear Regression** for predicting continuous target variables and evaluate the model performance using appropriate metrics.

NAME OF STUDENT	**Abhishek Yadav**
USN NO.	**CS23071**
Semester/Year	**06/03**
DATE OF PERFORMANCE	**17/01/2026**
DATE OF SUBMISSION	**24/01/2026**

AIM: To implement Linear Regression for predicting continuous target variables and evaluate the model performance using appropriate metrics.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- Develop a deeper understanding of the linear regression model and its limitations;
- Know how to diagnose and apply corrections to some problems with the generalized linear model found in real data; discussed;
- Use and understand generalizations of the linear model to binary and count data;
- Develop a greater familiarity with a range of techniques and methods through a diverse set of theoretical and applied readings;
- Know where to go to learn more about the techniques in this class and those called for that were not covered in this class.

THEORY:

Linear Regression:

It is the basic and commonly used type for predictive analysis. It is a statistical approach to modeling the relationship between a dependent variable and a given set of independent variables.

These are of two types:

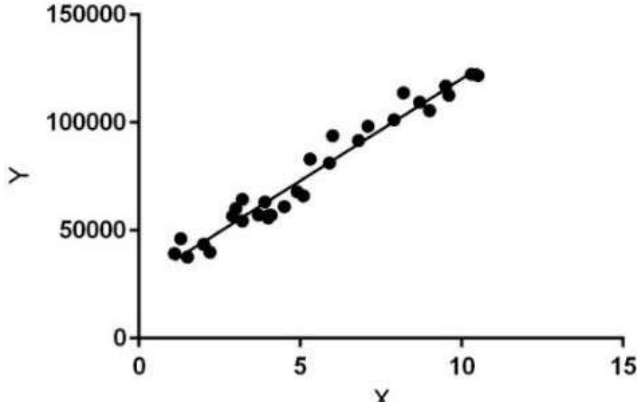
1) Simple linear Regression

2) Multiple Linear Regressions

Linear Regression is a machine learning algorithm based on supervised learning. It performs a regression task. Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting. Different regression models differ based on – the kind of relationship between dependent and independent variables they are considering, and the number of independent variables getting used. There are many names for a regression's dependent variable. It may be called an outcome variable,

criterion variable, endogenous variable, or regressand. The independent variables can be called exogenous variables, predictor variables, or regressors.

Linear regression is used in many different fields, including finance, economics, and psychology, to understand and predict the behavior of a particular variable. For example, in finance, linear regression might be used to understand the relationship between a company's stock price and its earnings, or to predict the future value of a currency based on its past performance.



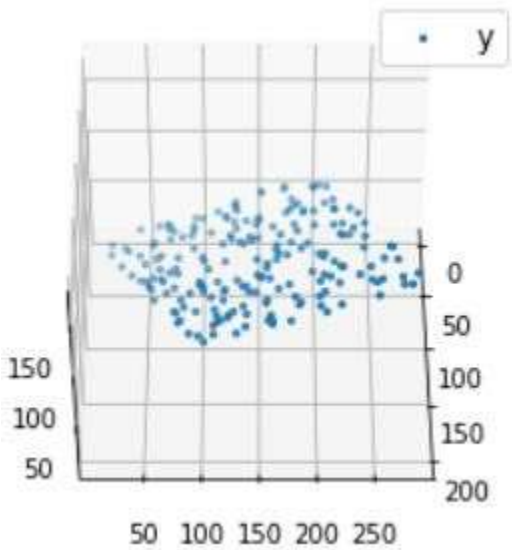
Linear regression performs the task to predict a dependent variable value (y) based on a given independent variable (x)). Hence, the name is Linear Regression. In the figure above, X (input) is the work experience and Y (output) is the salary of a person. The regression line is the best fit line for our model.

Sample Code(For reference only):

```
import numpy as np
import matplotlib as mpl
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

def generate_dataset(n):
    x = []
    y = []
    random_x1 = np.random.rand()
    random_x2 = np.random.rand()
    for i in range(n):
        x1 = i
        x2 = i/2 + np.random.rand()*n
        x.append([1, x1, x2])
        y.append(random_x1 * x1 + random_x2 * x2 + 1)
```

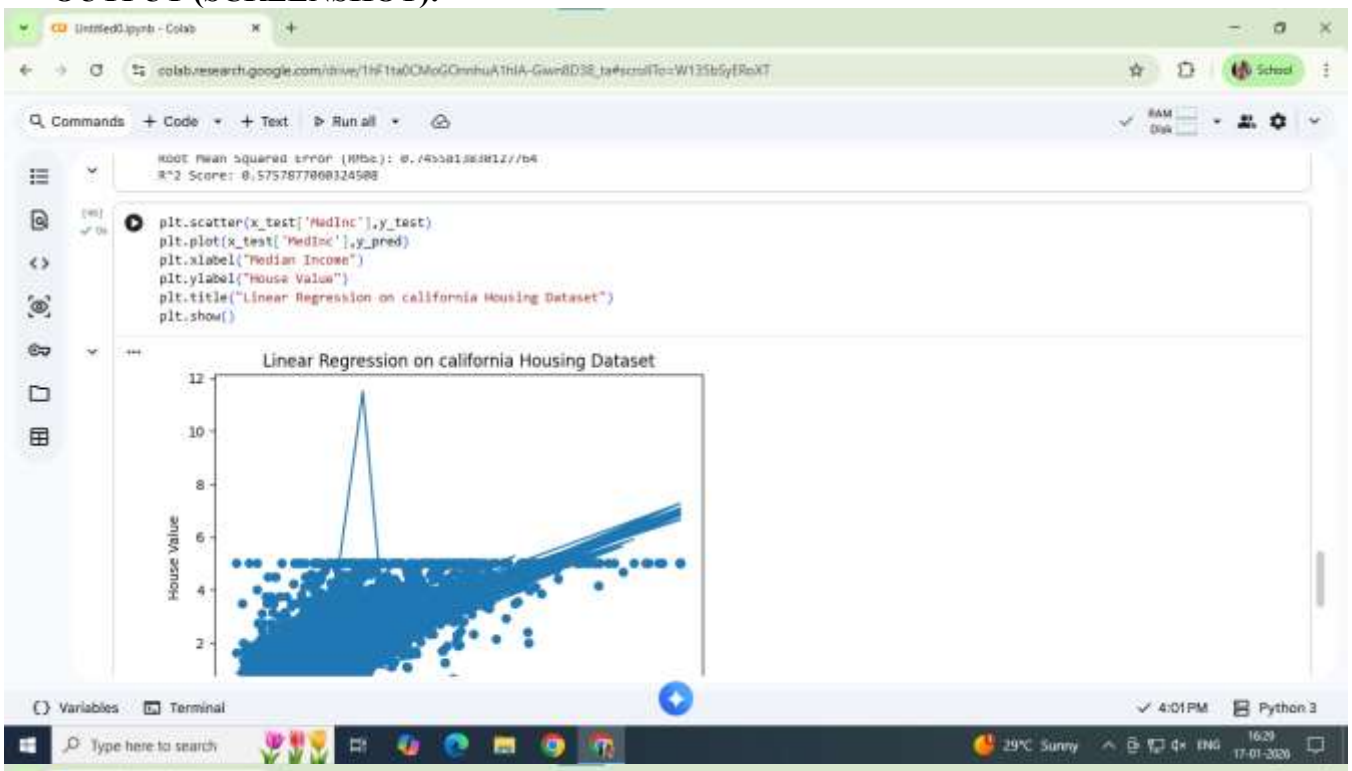
```
    return np.array(x), np.array(y)

x, y = generate_dataset(200)

mpl.rcParams['legend.fontsize'] = 12

fig = plt.figure()
ax = fig.add_subplot(projection ='3d')

ax.scatter(x[:, 1], x[:, 2], y, label ='y', s = 5)
ax.legend()
ax.view_init(45, 0)

plt.show()
```

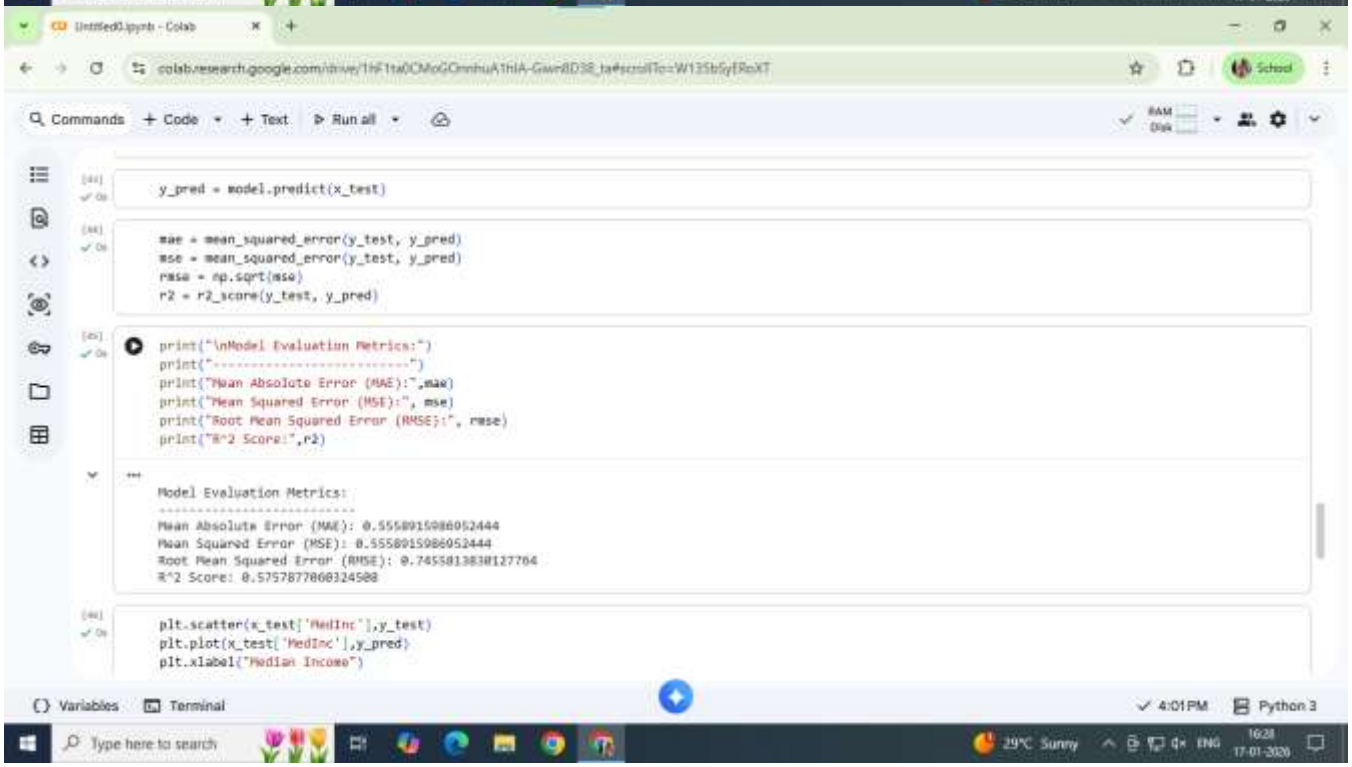
OUTPUT (SCREENSHOT):



PROGRAM CODE:

OUTPUT (SCREENSHOT):





```
x_train.shape
```

```
(16512, 8)
```

```
x_test.shape
```

```
(4128, 8)
```

```
y_test.shape
```

```
(4128,)
```

```
model = LinearRegression()
model.fit(x_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```
y_pred = model.predict(x_test)
```

```
mae = mean_squared_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
x_train
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117.03
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118.16
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120.48
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117.11
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119.80
...	...	...	...	...	...	...	...	...
11284	6.3700	35.0	6.129032	0.926267	658.0	3.032258	33.78	-117.96
11964	3.0500	33.0	6.868597	1.269488	1753.0	3.904232	34.02	-117.43
5390	2.9344	36.0	3.986717	1.079696	1756.0	3.332068	34.03	-118.38
860	5.7192	15.0	6.395349	1.067979	1777.0	3.178891	37.58	-121.96
15795	2.5755	52.0	3.402576	1.058776	2619.0	2.108696	37.77	-122.42

16512 rows × 8 columns

```
y
```

	target
0	4.526
1	3.585
2	3.521
3	3.413
4	3.422
...	...
20635	0.781
20636	0.771
20637	0.923
20638	0.847
20639	0.894

20640 rows x 1 columns

dtype: float64

max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010

```
x = df.drop('target', axis=1)
y = df['target']
```

```
x
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109642	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25
...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24

```
   1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
   2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
   3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
   4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  target
0    -122.23   4.526
1    -122.22   3.585
2    -122.24   3.521
3    -122.25   3.413
4    -122.25   3.422
```

```
df.describe()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	target
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.149990
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.196000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.797000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.647250
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010

Double-click (or enter) to edit

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df['target'] = housing.target
print(df.head())
```

```
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  target
```

CONCLUSION:

DISCUSSION AND VIVA VOCE:

1. When would you choose Linear Regression over more complex models in a real project?
2. How do you check whether Linear Regression assumptions hold in real-world data?
3. How do you handle multicollinearity in Linear Regression for industry datasets?
4. How do evaluation metrics (MAE, RMSE, R^2) influence business decisions in regression problems?
5. Why is feature scaling sometimes ignored in Linear Regression but critical in some industry cases?
6. How do you decide whether a Linear Regression model is good enough for deployment?

REFERENCE:

- https://www.google.com/search?q=linear+regression+machine+learning&oq=Linear+Regression&aqs=chrome.1.0i433i512j0i512j69i59j0i131i433i512l2j69i60l3.4688j0j7&sourceid=chrome&ie=UTF-8
- https://pubs.acs.org/doi/10.1021/acsomega.2c00362
- https://colab.research.google.com/drive/1gbyjW-RgdgVkUNyRkHAWmjMr7RUCUrYX